



devicetree
.org

DeviceTree Structure

Root node

Node (child of root node)

Sub-Node (child of *leds* node)

Node label (e.g. *led3*)

Property (e.g. *gpios*)

```
nrf54l15dk_common.dtsi X
C > Nordic > NCS > v3.2.1 > zephyr > boards > nordic > nrf54l15dk > nrf54l15dk_common.dtsi > leds
#include "nrf54l15dk_nrf54l_05_10_15-pinconf.dtsi"

8
9 / {
10     leds {
11         compatible = "gpio-leds";
12
13         led0: led_0 {
14             gpios = < &gpio2 9 GPIO_ACTIVE_HIGH >;
15             label = "Green LED 0";
16         };
17
18         led1: led_1 {
19             gpios = < &gpio1 10 GPIO_ACTIVE_HIGH >;
20             label = "Green LED 1";
21         };
22
23         led2: led_2 {
24             gpios = < &gpio2 7 GPIO_ACTIVE_HIGH >;
25             label = "Green LED 2";
26         };
27
28         led3: led_3 {
29             gpios = < &gpio1 14 GPIO_ACTIVE_HIGH >;
30             label = "Green LED 3";
31         };
32     };
33
34     pwmleds {
```

DeviceTree Handling

DeviceTree File:

```
/*  
 *  
 * led0: led_0 {  
 *     gpio0 = <gpio2 9 GPIO_ACTIVE_HIGH>;  
 *     label = "Green LED 0";  
 * };  
 *  
 * led1: led_1 {  
 *     gpio1 = <gpio1 10 GPIO_ACTIVE_HIGH>;  
 *     label = "Green LED 1";  
 * };  
 *  
 * led2: led_2 {  
 *  
 * }  
 */  
leds {  
    compatible = "gpio-leds";  
    led0: led_0 {  
        gpio0 = <gpio2 9 GPIO_ACTIVE_HIGH>;  
        label = "Green LED 0";  
    };  
    led1: led_1 {  
        gpio1 = <gpio1 10 GPIO_ACTIVE_HIGH>;  
        label = "Green LED 1";  
    };  
    led2: led_2 {  
    }  
};
```

① Point into DeviceTree Node

② Get the Property „gpio0“

③ Use DeviceTree info to
configure GPIO

C Code:

```
/* The devicetree node identifier for the "led0". */  
#define LED0_NODE DT_NODENAME(led0) ①  
  
/*  
 * A build error on this line means your board is unsupported.  
 * See the sample documentation for information on how to fix this.  
 */  
static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(LED0_NODE, gpio0); ②  
  
32 ret = gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);  
33 if (ret < 0) {  
34     return 0;  
}
```

How to obtain a Node Identifier

Examples on how to obtain the Node Identifier for `uart@abcd1234` node

DTS

```
/dts-v1/;
/ {
    chosen {
        zephyr,console = &uart0;
    };
    aliases {
        my-uart = &uart0;
    };
    soc {
        uart0: uart@abcd1234 {
            compatible = „vnd,soc-uart“;
            reg = <0xabcd1234 0x1000>;
            current-speed = <9600>;
        };
    };
};
```

```
/* by full path */
DT_PATH(soc, uart_abcd1234)
```

```
/* by node label */
DT_NODELABEL(uart0)
```

```
/* by using an alias */
DT_ALIAS(my_uart)
```

```
/* by using a choice */
DT_CHOSEN(zephyr_console)
```

```
/* by specifying an instance number of a
 * given compatible, e.g. 0 */
```

```
DT_INST(0, vnd_soc_uart)
```

C

Node Identifiers

Important

Non-alphanumeric characters like dash (-) and the at sign (@) in devicetree names are converted to underscores (_). The names in a DTS are also converted to lowercase.

https://docs.nordicsemi.com/bundle/ncs-latest/page/zephyr/build/dts/api-usage.html#node_identifiers

Example:

nrf52840dk_nrf52840.dts file

```
/* These aliases are provided for compatibility with samples */
aliases {
    led0 = &led0;
    led1 = &led1;
    led2 = &led2;
    led3 = &led3;
    pwm-led0 = &pwm_led0;
    sw0 = &button0;
    sw1 = &button1;
    sw2 = &button2;
    sw3 = &button3;
    bootloader-led0 = &led0;
    mcuboot-button0 = &button0;
    mcuboot-led0 = &led0;
    watchdog0 = &wdt0;
```

zephyr/samples/basic/blinky_pwm

```
#include <zephyr/drivers/pwm.h>

static const struct pwm_dt_spec pwm_led0 = PWM_DT_SPEC_GET(DT_ALIAS(pwm_led0));

#define MIN_PERIOD PWM_SF(10) / 1280
```

Accessing Properties from C Code

Examples on how to access DeviceTree Information

DTS

```
/dts-v1/;
/ {
    ...
    i2c0: i2c@abcd1234 {
        ...
        sensor0: sensor@ff {
            compatible = "vnd,mysensor";
            reg = <0xff>;
            int-gpios = <&gpio0 1
                        GPIO_ACTIVE_LOW>;
            sample-freq = <1000>;
        };
    };
};
};
```

C

```
/* use Node Label to obtain sensor node
 * identifier */
#define SENSOR_0 DT_NODELABEL(sensor0)

/* expands to 0xff */
DT_REG_ADDR(SENSOR_0)

/* expands to gpio() node identifier */
DT_PHANDLE(SENSOR_0, int_gpios)
/* equivalent (GPIO specific helpers) */
DT_GPIO_CTRL(SENSOR_0, int_gpios)

/* expands to 1 (pin cell) */
DT_PHA(SENSOR_0, int_gpios, pin)
/* equivalent (GPIO specific helpers) */
DT_GPIO_PIN(SENSOR_0, int_gpios)

/* expands to 1000 */
DT_PROP(SENSOR_0, sample_freq)
```

Most Important DeviceTree Properties

Property	
compatible	Name of the hardware device the node represents. Recommended format: vendor,device Build system uses compatible property to find the right <u>bindings</u> for the node.
reg	Information used to address the device. Its value is specific to a device. In general, it is a sequence of (<i>address</i> , <i>length</i>) pairs.
status	String describes whether the node is enabled. "okay" for enabled, and "disabled" for disabled.

DeviceTree Property Types

Property type	How to write	Example
string	Double quoted	<code>a-string = "hello, world!";</code>
int	between angle brackets (<code><</code> and <code>></code>)	<code>an-int = <1>;</code>
boolean	for <code>true</code> , with no value (for <code>false</code> , use <code>/delete-property/</code>)	<code>my-true-boolean;</code>
array	between angle brackets (<code><</code> and <code>></code>), separated by spaces	<code>foo = <0xdeadbeef 1234 0>;</code>
uint8-array	in hexadecimal <i>without</i> leading <code>0x</code> , between square brackets (<code>[</code> and <code>]</code>).	<code>a-byte-array = [00 01 ab];</code>
string-array	separated by commas	<code>a-string-array = "string one", "string two", "string three";</code>
phandle	between angle brackets (<code><</code> and <code>></code>)	<code>a-phandle = <&mynode>;</code>
phandles	between angle brackets (<code><</code> and <code>></code>), separated by spaces	<code>some-phandles = <&mynode0 &mynode1 &mynode2>;</code>
phandle-array	between angle brackets (<code><</code> and <code>></code>), separated by spaces	<code>a-phandle-array = <&mynode0 1 2>, <&mynode1 3 4>;</code>

DeviceTree Bindings

- **Content** of DeviceTree, including data types, is **described** in **binding** files
- Binding files are used by the DeviceTree parser to **validate content**
- Binding files are written in **YAML**, structure custom to Zephyr

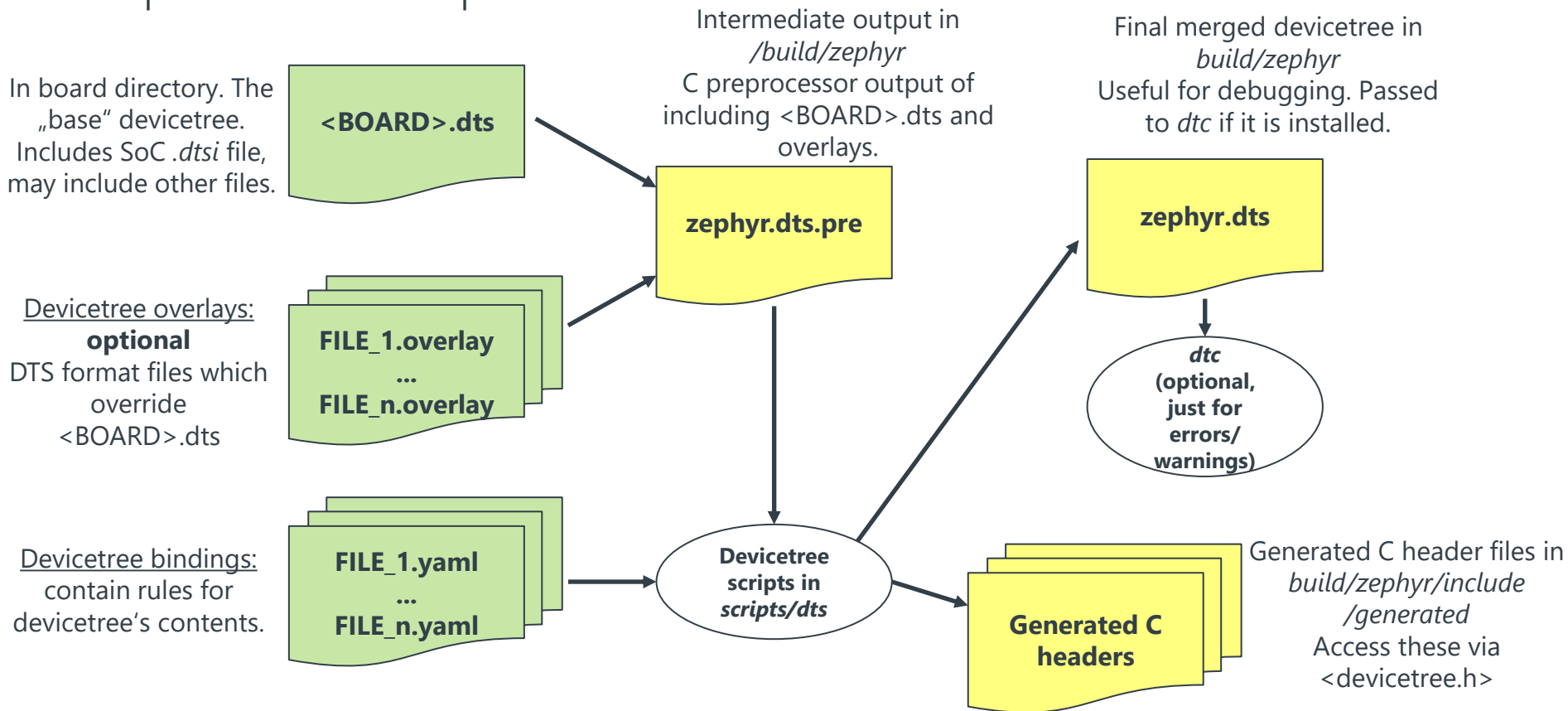
Binding File:

```
description: my dev driver
compatible: vnd,dev
include: base.yaml
properties:
  foo:
    type: int
    required: true
```

DeviceTree Source File:

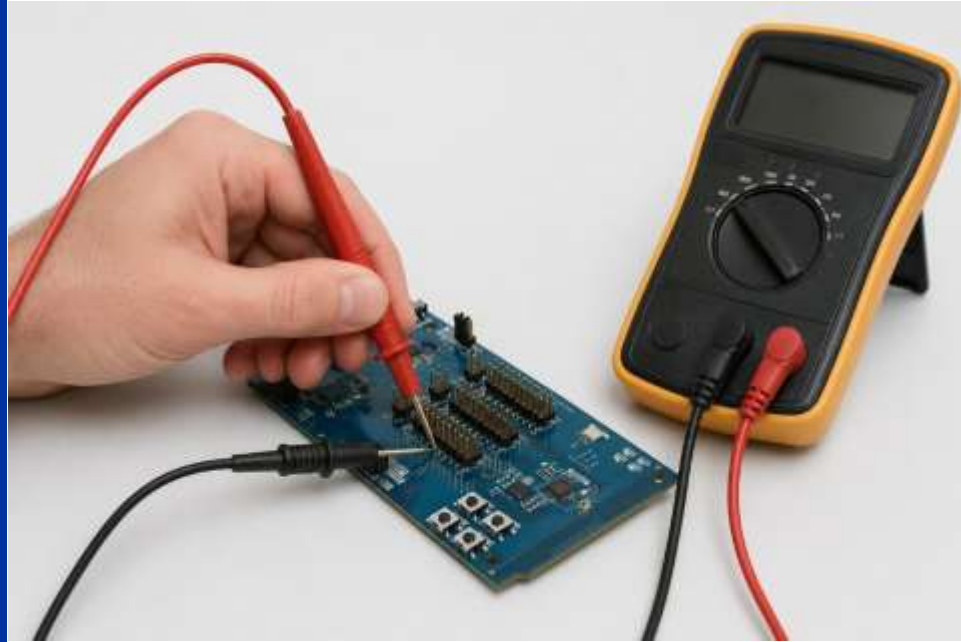
```
/ {
    dev0: dev@01234 {
        compatible = „vnd,dev“
        reg = <0x01234>;
        foo = <7>;
    };
};
```

Input and Output Files



Hands-on

- DeviceTree
 - Explore: Reading a Real DeviceTree
 - Extend: Overlays and Custom Bindings
 - Debug: Fixing a Broken DeviceTree





Step 1 - Download, Add project to VS code, and Build the workshop application

1. Download the lab project from [here](#)
2. Add this project with **Open an existing application** to VS code.
3. **Add build configuration** and use following configuration:
Board Target: nrf54l15dk/nrf54l15/cpuapp
4. **Generate and Build** the project.
5. **Flash** the project. Open Serial Terminal and observe the boot log.



Step 2 - Explore generated DeviceTree

6. Open **build/dt_workshop_app/zephyr/zephyr.dts**.

➤ Search for **led0** => What is the DeviceTree Node path? Which GPIO is used for LED0?

➤ Search for **button0** => What is the DeviceTree node path? Which GPIO is used for Button0?

7. Search for **uart20** and verify the configured baud rate (**current-speed**) and node **status**.

💡 **Tip:** This is the DK's console UART (**chosen zephyr,console**).

8. Open **build/dt_workshop_app/zephyr/include/generated/zephyr/devicetree_generated.h**

9. Search for the GPIO pin associated with one of the LEDs (e.g. search "led_0" and find your LED's ordinal, take a look on the following "led_0" definitions) and observe how DeviceTree information has been converted into generated macro.

💡 This is what DT_ macros expand to at compile time - there is no runtime devicetree parsing in Zephyr.



Step 3 – Match the code to the tree

9. Open **dt_workshop_app/src/main.c**. For each `DT_ALIAS` and `DT_NODELABEL` macro:
 - Identify the corresponding node in `zephyr.dts`.
 - Explain which DeviceTree node it resolves to.
 - Verify that the serial output matches the DeviceTree configuration.



Step 4 – Bonus: Visual Tools

- In VS Code with the nRF Connect extension, hover over a node or property in an overlay file to see live binding documentation.
- Use **DeviceTree Visual Editor** and check LED and button pins.



Step 5 - Explore the overlay

10. Open **dt_workshop_app/boards/nrf54l15dk_nrf54l15_cpuapp.overlay**

Observe that it:

- creates the alias `my-led`
- adds the custom node `fake_sensor0`



Step 6 - Inspect the custom binding

11. Open **dt_workshop_app/dts/bindings/sensor/workshop,fake-sensor.yaml**

Review the defined properties and their validation rules.

- 💡 Zephyr automatically searches **/dts/bindings** for bindings, no extra CMake configuration required.

Note the three properties:

- **sensor-label** (required string),
- **sampling-frequency-hz** (int, default 10), and
- **io-gpios** (a phandle-array, the same property "type" used by real GPIO-based drivers).



Step 7 – Verify

12. In **build/dt_workshop_app/zephyr/zephyr.dts**, find `fake-sensor@0` and confirm your properties are present.
13. Verify the serial output displays:
 - the `my-led` alias
 - the `sensor-label`
 - the sample frequency



Step 8 - Modify the configuration

14. Change the value of `sampling-frequency-hz`

Rebuild and verify that the updated value appears in the serial output without modifying the application source code.

15. Add a second alias: `my-button = &button0;`

Update **main.c** to print the alias using: `DT_ALIAS(my_button)`

16. Temporarily delete the sensor-label line from the overlay and rebuild.

Read the error message. It should point you straight at the missing required property - this is edtlib enforcing your binding's `required: true`. Put the line back before continuing.

💡 Edtlib (edtlib.py) is a Python library used to parse and interpret hardware DeviceTrees by combining DeviceTree source files with YAML binding files. It provides a higher-level view of system hardware elements like buses and interrupts, and is widely used in the Zephyr RTOS build system



Step 9 – Open new project

1. Download the lab project from [here](#)
2. Add the project to VS Code. **Add build configuration** with board target

nrf54l15dk/nrf54l15/cpuapp



Step 10 – Build and Fix Issues

20. Read the first error message end-to-end before touching anything. It will reference boards/nrf54l15dk_nrf54l15_cpuapp.overlay.
21. Fix only that one problem, then rebuild.
22. Repeat the process until all DeviceTree errors have been resolved.



Tip: During this process, expect to encounter:

- an undefined DeviceTree label
- a property with an incorrect type
- an incompatible compatible string

20. Once the build succeeds, flash and check the serial log for: "Both DT-described devices report ready!". If you see it, your fixes are correct - a clean build alone does not guarantee that.

```
*** Booting nRF Connect SDK v3.4.0-99553055607b ***
*** Using Zephyr OS v4.4.0-bf801e4e3d19 ***

=== DeviceTree Debugging Exercise ===
weather0: label=Workshop-Weather-Node freq=5Hz pin=7
backup0 : label=Backup-Node freq=5Hz pin=10
Both DT-described devices report ready. Overlay is fixed!
█
```